

OrderScan Test Register

Exactly which backend endpoints have tests, which do not, and the pattern to follow when writing the rest. Read this before adding any test to this repo.

As of 16 Sept 2026 Commit 86a4dfa 342 tests passing tsc clean

65 of 120 server functions covered • 10 of 25 files

55 remaining

Correct an earlier miscount. Some notes in this project say “151 server functions”. That figure counted every occurrence of `createServerFn`, including import lines. The real number, counting `export const X = createServerFn(`, is **120**.

Why a handler must be split before it can be tested

A handler written inline inside `createServerFn(...).handler(...)` cannot be called from a test. The `requireSupabaseAuth` middleware calls `getRequest()`, which only resolves inside a real HTTP request. So every handler is written as a plain function with a one-line wrapper under it:

```
import type { ServerContext } from "../server-context";

export async function doThingCore(data: Input, context: ServerContext) {
  /* all the logic lives here */
}

export const doThing = createServerFn({ method: "POST" })
  .middleware([requireSupabaseAuth])
  .inputValidator((input: Input) => input)
  .handler(({ data, context }) => doThingCore(data, context));
```

The test imports `doThingCore`. A handler taking no `data` becomes `doThingCore(context)`.

The mock

`src/lib/test-utils/mock-supabase.ts` queues one canned result per `.from(table)` call, first-in-first-out per table name. A call with nothing left queued **throws** — an incomplete setup fails loudly instead of quietly reaching a real client. That throw is also useful as an assertion: queue nothing for a table to prove the code never touches it.

```
const { client, queueResponse, queueError, allowRole, getChain, rpc } = createMockSupabase();

queueResponse("profiles", { data: { active_workspace_id: "ws1" }, error: null });
allowRole(); // has_workspace_role → true
allowRole(false); // → assert the refusal
queueError("stores", "boom"); // a failed query

await doThingCore({ id: "s1" }, { supabase: client as never, userId: "u1" });
```

```
expect(getChain("stores").eq).toHaveBeenCalledWith("workspace_id", "ws1");
```

Almost every test needs `allowRole()`. `assertActiveWorkspaceRole` calls `rpc("has_workspace_role")`, so without it no test gets past the authorization check. It also reads `profiles` first for the active workspace — queue that too.

`rpc()` is both awaitable and chainable. Some callers await it directly; others append `.maybeSingle()` (a set-returning SQL function). Both work.

10 files, 65 endpoints, 190 tests

FILE	ENDPOINTS	TESTS	FOCUS OF THE COVERAGE
billing.functions.ts	13	30	Pricing from real payments, auto-apply, refusals
workspace-members.functions.ts	11	20	Every role guard, owner protection, audit rows
commission.functions.ts	10	29	Band overlap, month totals, workspace scoping
brand.functions.ts	9	22	Revenue vs cost split, empty months, refusals
reconcile.functions.ts	6	18	Owed vs paid, settlement states, scoping
payouts.functions.ts	5	16	Generation, skip-if-paid, failure reporting
stores.functions.ts	4	18	Normalisation, duplicates, not-found guard
admin-recovery.functions.ts	3	20	The recovery guard itself, owner takeover
queue.functions.ts	3	12	Claim/retry decisions, staleness
batch-actions.functions.ts	1	5	Partial upload failure, all-fail cleanup
10 files	65	190	

A further 124 tests cover pure functions with no database: `plans` (23), `template-extract` (27), `slab-validation` (15), `plan-features` (15), `brand-slabs` (15), `format` (14), `db-payload` (8), `employee-pay` (7). Plus 28 Python tests for the screenshot-reading service.

15 files, 55 endpoints, in the order to do them

Ordered by what a mistake would cost. Everything above the line has already been done; this is what is left.

FILE	ENDPOINTS	PRIORITY	WHY IT MATTERS
employees.functions.ts	12	NEXT	Salary, advances, plan seat limits. Two bugs already found here.
partners.functions.ts	6	NEXT	Partner records, plan partner limits. One bug already found.
month-close.functions.ts	4	NEXT	Locks a month against edits. Interacts with a database trigger.
workspace.functions.ts	6	THEN	Creating, renaming and deleting a whole workspace.
admin-users.functions.ts	1	THEN	Super-admin surface.
reliability.functions.ts	9	THEN	Report generation and file delivery.
notifications.functions.ts	5	THEN	In-app alerts and preferences.
statements.functions.ts	2	THEN	Partner statements.
portal.functions.ts	2	THEN	What a partner sees of their own data.
performance.functions.ts	2	LATER	Store and partner performance figures.
usage.functions.ts	2	LATER	Plan usage counters.
kpis.functions.ts	1	LATER	Dashboard tiles.
trends.functions.ts	1	LATER	Dashboard charts.
activity.functions.ts	1	LATER	Activity feed.
screenshot-upload.functions.ts	1	LATER	Signed upload URLs.
15 files	55		

Also untested: the 10 route handlers under `src/routes/api/`, and every React component — `vitest.config.ts` sets `environment: "node"`, so there is no DOM available for component tests today.

The five defects that kept recurring

33 problems were found across the covered files, and they clustered into five shapes. Write the test for the shape, not just the happy path.

1. **Missing workspace filter.** Every query and every write must carry `.eq("workspace_id", wsId)`. Assert it with `getChain(table).eq`. Found 7 times.
2. **Write by id alone.** An update or delete matching only `.eq("id", ...)` lets one company touch another's row. Found 5 times.
3. **Unchecked write result.** `.select("id")` and treat zero rows as a failure — otherwise a refused write reports success. Found 5 times.
4. **Discarded query error.** `const { data } = await ...` without touching `error` turns a failure into an empty result. On a money report that means a confident zero. Found 4 times.
5. **Missing role check.** Every write endpoint needs an `allowRole(false)` test. One listing endpoint had no check at all.

A checklist per endpoint

- Refusal: `allowRole(false)` → throws.
- Scoping: the workspace filter is actually applied.
- Not found: a row from another workspace → reported, not silently ok.
- Error path: `queueError(...)` → surfaces, not swallowed.
- Edge values: `null`, `0`, empty list, division by zero.

Commands and file locations

```
npm test                                # vitest — 314 tests
npm run test:watch
npx vitest run src/lib/stores.functions.test.ts
npx tsc --noEmit -p tsconfig.json      # must stay clean

pip install -r ocr-service/requirements/test.txt # once
pytest                                # 28 tests, works from the repo root
```

Tests live beside the file they cover: `src/lib/x.ts` → `src/lib/x.test.ts`.

Shared pieces are `src/lib/test-utils/mock-supabase.ts` and `src/lib/server-context.ts`. The Python suite lives in `ocr-service/tests/`.

Install `requirements/test.txt`, not `dev.txt`. The dev file pulls in `base.txt` and therefore `paddlepaddle`, which is a large download and will not install on Windows. The suite never needs it — `conftest.py` keeps the lazy `paddle` import from ever running.

Two traps that cost real time here. Scripted edits that `sys.exit()` on a failed match *before* saving will silently discard edits that already matched — one fix was reported as applied and never written to disk, and only a test caught it. Collect every edit, verify all match, then write once. And check `tsc` before committing, not after: one commit went out with two type errors in a test file.